

Module 3 Lab: Version Control and Collaborative Tooling

Create a GitHub Repository

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

wongellakew-lab

Repository name *

M3-Version control and collaborative toolin

✔ Your new repository will be created as M3-Version-control-and-collaborative-toolin. The repository name can only contain ASCII letters, digits, and the characters ., -, and _.

Great repository names are short and memorable. How about [laughing-adventure](#)?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Initialize git

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git init
Initialized empty Git repository in C:/Users/wonge/Desktop/FullStackDevelopment/Version Control/.git/
```

Create .gitignore

```
.gitignore
node_modules/
dist/
*.js
*.js.map
*d.ts
!src/**/*.d.ts
```

Git add and status

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git add .
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   index.ts
    new file:   models/api-response.model.ts
    new file:   models/assessment.model.ts
    new file:   models/course.model.ts
    new file:   models/enrollment.model.ts
    new file:   models/student.model.ts
    new file:   package.json
    new file:   tsconfig.json
```

Commit

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit -m "feat: Setup github + push"
[main (root-commit) d76ec4c] feat: Setup github + push
 9 files changed, 382 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 index.ts
 create mode 100644 models/api-response.model.ts
 create mode 100644 models/assessment.model.ts
 create mode 100644 models/course.model.ts
 create mode 100644 models/enrollment.model.ts
 create mode 100644 models/student.model.ts
 create mode 100644 package.json
 create mode 100644 tsconfig.json
```

Connect to GitHub and push

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git branch -M main
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git push -u origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 4.50 KiB | 1.12 MiB/s, done.
Total 12 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/wongellakew-lab/M3-Version-control-and-collaborative-tooling.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Verify on GitHub

The screenshot shows the GitHub interface for a repository named "M3-Version-control-and-collaborative-tooling". The repository is public and has 1 branch (main) and 0 tags. The commit history shows a single commit by user "wongellakew-lab" with the message "feat: Setup github + push" at commit hash "d76ec4c", made 12 minutes ago. The commit details show 9 files changed: "models", ".gitignore", "index.ts", "package.json", and "tsconfig.json", all with the message "feat: Setup github + push" and committed 12 minutes ago.

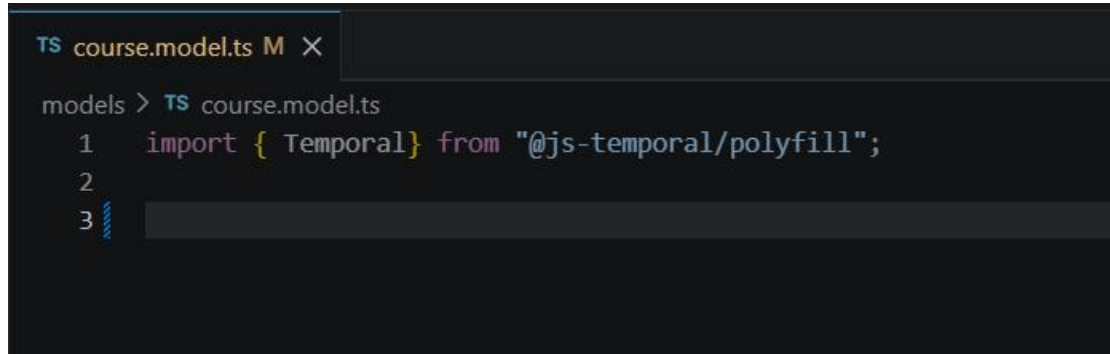
File	Commit Message	Time
models	feat: Setup github + push	12 minutes ago
.gitignore	feat: Setup github + push	12 minutes ago
index.ts	feat: Setup github + push	12 minutes ago
package.json	feat: Setup github + push	12 minutes ago
tsconfig.json	feat: Setup github + push	12 minutes ago

Exercise 1: Switch and restore

Create a Feature Branch

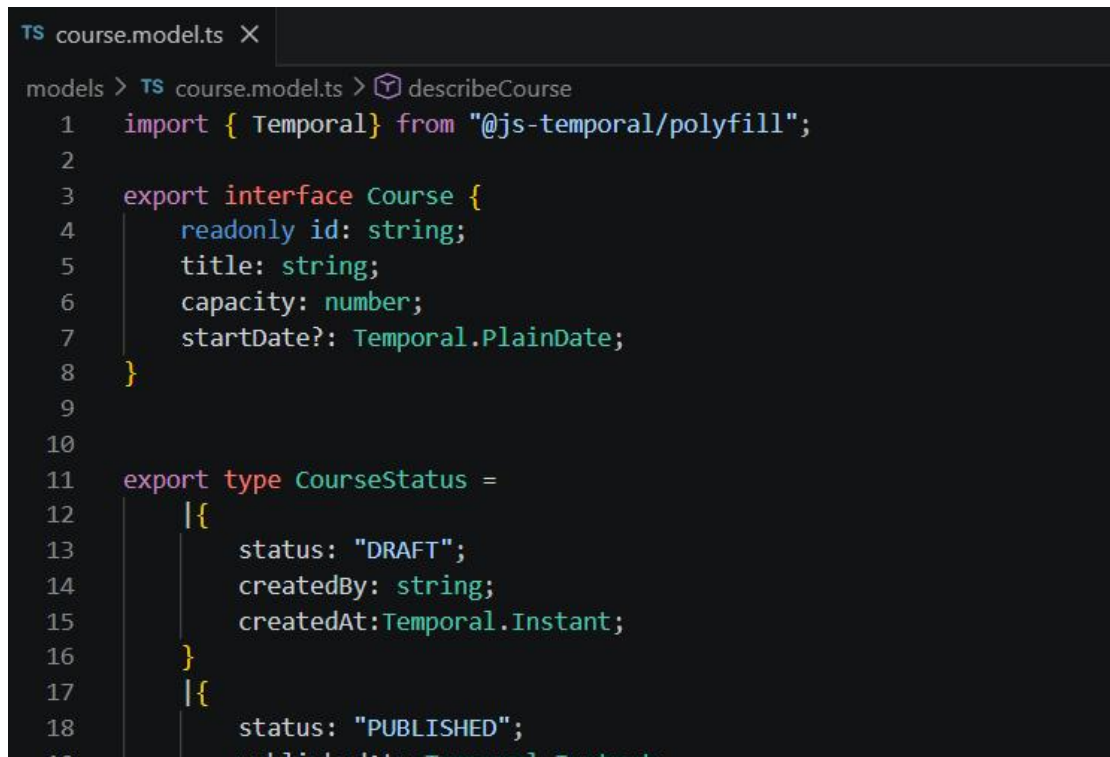
```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git switch -c feature/course-capacity
Switched to a new branch 'feature/course-capacity'
```

Make changes to course.models.ts file



```
TS course.model.ts M X
models > TS course.model.ts
1  import { Temporal } from "@js-temporal/polyfill";
2
3  
```

After git restore models/course.models.ts



```
TS course.model.ts X
models > TS course.model.ts > describeCourse
1  import { Temporal } from "@js-temporal/polyfill";
2
3  export interface Course {
4      readonly id: string;
5      title: string;
6      capacity: number;
7      startDate?: Temporal.PlainDate;
8  }
9
10
11 export type CourseStatus =
12     | {
13         status: "DRAFT";
14         createdBy: string;
15         createdAt: Temporal.Instant;
16     }
17     | {
18         status: "PUBLISHED";
19         publishedAt: Temporal.Instant;
```

Exercise 2: Detached HEAD and relog

Create a normal commit

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit --allow-empty -m "feat: adding basic student logic"
[main b64fb24] feat: adding basic student logic
```

Detach HEAD(go back one commit without a branch)

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git switch --detach HEAD~1
HEAD is now at d76ec4c feat: Setup github + push
```

Commit work while on detached

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit --allow-empty -m "feat: certificate generation logic"
[detached HEAD fc853f4] feat: certificate generation logic
```

Switch back and check history

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git switch main
Warning: you are leaving 1 commit behind, not connected to
any of your branches:

fc853f4 feat: certificate generation logic
```

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git log --oneline
b64fb24 (HEAD -> main) feat: adding basic student logic
d76ec4c (origin/main, feature/course-capacity) feat: Setup github + push
```

Recover with relog

View the hidden journal of every HEAD movement

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git relog
b64fb24 (HEAD -> main) HEAD@{0}: checkout: moving from fc853f499e9d38d4facb520c4bf9aab5ec425c61 to main
fc853f4 HEAD@{1}: commit: feat: certificate generation logic
d76ec4c (origin/main, feature/course-capacity) HEAD@{2}: checkout: moving from main to HEAD~1
b64fb24 (HEAD -> main) HEAD@{3}: commit: feat: adding basic student logic
d76ec4c (origin/main, feature/course-capacity) HEAD@{4}: checkout: moving from feature/course-capacity to main
```

Rescue the lost commit by creating a new branch pointing at the orphaned commit and verify

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git branch rescue-certificate fc853f4
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git switch rescue-certificate
Switched to branch 'rescue-certificate'
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git log --oneline
fc853f4 (HEAD -> rescue-certificate) feat: certificate generation logic
d76ec4c (origin/main, feature/course-capacity) feat: Setup github + push
```

Exercise 3: Atomic Commits

Switch to main branch

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
```

Create a new branch named "feature/enrollment-capacity"

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git switch -c feature/enrollment-capacity
Switched to a new branch 'feature/enrollment-capacity'
```

Create a new file named "EnrollmentValidator.ts" inside models/

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git status
On branch feature/enrollment-capacity
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    models/EnrollmentValidator.ts
```

Stage the file

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git status
On branch feature/enrollment-capacity
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   models/EnrollmentValidator.ts
```

Commit using Conventional commit format

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit -m "Feat(enrollment): add max capacity validator"
[feature/enrollment-capacity 0bc8235] feat(enrollment): add max capacity validator
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 models/EnrollmentValidator.ts
```

Exercise 4: The merge conflict (Collaborative Integration)

Create a baseline file, stage and commit

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> echo "export const PASS_THRESHOLD = 60;"> assessment-config.ts
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git add .\assessment-config.ts
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit -m "chore: setup baseline assessment rules"
[main ff3b20d] chore: setup baseline assessment rules
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 assessment-config.ts
```

Simulate a colleague's change on a separate branch

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git switch -c feature/strict-assessments
Switched to a new branch 'feature/strict-assessments'
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> echo "export const PASS_THRESHOLD = 75;"> assessment-config.ts
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git add .\assessment-config.ts
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit -m "fix(assessment): increase threshold for honors certification"
[feature/strict-assessments a9a4b8a] fix(assessment): increase threshold for honors certification
1 file changed, 0 insertions(+), 0 deletions(-)
```

Parallel change on main

```
74ced80 (HEAD -> main) fix(assessment): lower threshold for inclusive grading
ff3b20d chore: setup baseline assessment rules
```

The collision

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git merge feature/strict-assessments
warning: Cannot merge binary files: assessment-config.ts (HEAD vs. feature/strict-assessments)
Auto-merging assessment-config.ts
CONFLICT (content): Merge conflict in assessment-config.ts
Automatic merge failed; fix conflicts and then commit the result.
```

Resolve the conflict and then stage the resolved files and commit

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git add .\assessment-config.ts
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control>
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit -m "Merge branch 'feature/strict-assessments' into main"
[main bc6235d] Merge branch 'feature/strict-assessments' into main
```

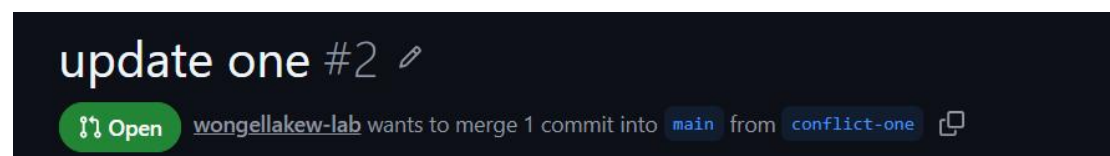
Verify

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git log --oneline --graph -5
*   bc6235d (HEAD -> main) Merge branch 'feature/strict-assessments' into main
| \
| * a9a4b8a (feature/strict-assessments) fix(assessment): increase threshold for honors certification
| * 74ced80 fix(assessment): lower threshold for inclusive grading
|/
* ff3b20d chore: setup baseline assessment rules
* b64fb24 feat: adding basic student logic
```

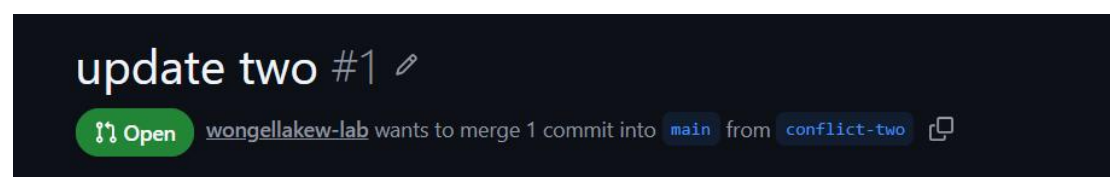
To simulate collisions in real-world team workflow using pull requests create two branches named “conflict-one” and “conflict-two” and edit the same file “assessment.config.ts” and push branches.

Open pull request for each branch

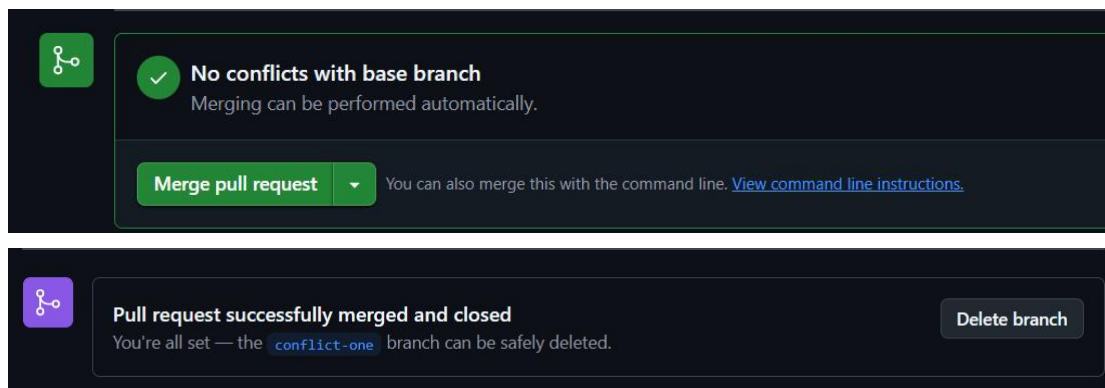
For branch conflict-one



For branch conflict-two

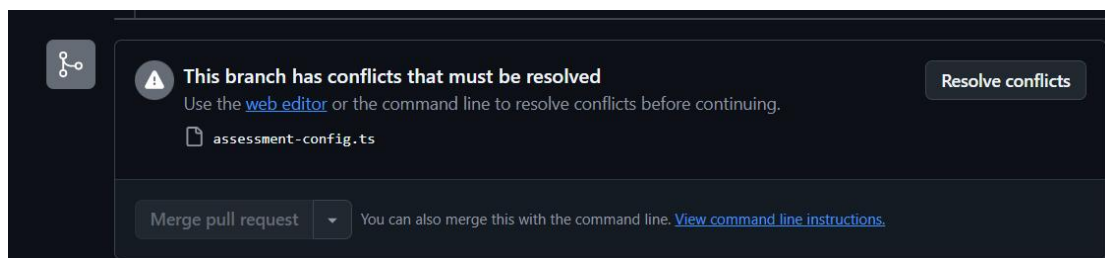


Merge the first PR(conflict-one)



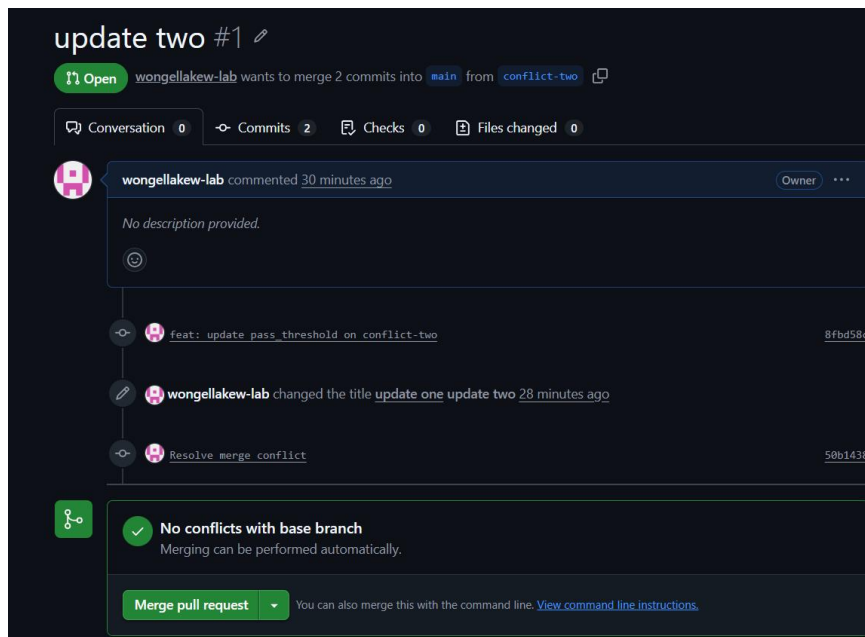
The screenshot shows two panels from the GitHub web interface. The top panel, titled 'No conflicts with base branch', indicates that merging can be performed automatically. It features a green checkmark icon and a 'Merge pull request' button. Below the button, it says 'You can also merge this with the command line. [View command line instructions.](#)'. The bottom panel, titled 'Pull request successfully merged and closed', informs the user that the pull request has been merged and the 'conflict-one' branch can be safely deleted. It includes a 'Delete branch' button.

Conflict on the second PR(conflict-two)



The screenshot shows a GitHub pull request page with a conflict. A warning icon and the text 'This branch has conflicts that must be resolved' are prominent. It suggests using the 'web editor' or the command line to resolve conflicts. A file named 'assessment-config.ts' is listed as being in conflict. A 'Resolve conflicts' button is visible in the top right. At the bottom, there is a 'Merge pull request' button and a link to 'View command line instructions'.

After repeating the steps used to resolve the conflict on the local machine we can merge with no issue.



The screenshot shows a GitHub pull request page for 'update two #1'. The pull request is from 'wongellakew-lab' and wants to merge 2 commits into 'main' from 'conflict-two'. The page shows a conversation with a comment from 'wongellakew-lab' stating 'No description provided.' Below the comment, the commit history is shown: 'feat: update_pass_threshold on conflict-two' (8fbd58c), 'wongellakew-lab changed the title update one update two 28 minutes ago', and 'Resolve merge conflict' (50b1438). At the bottom, a green checkmark and the text 'No conflicts with base branch' indicate that merging can be performed automatically. A 'Merge pull request' button is present.

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git log --online --graph -5
* 50b1438 (HEAD -> conflict-two, origin/conflict-two) Resolve merge conflict
|
| * d68d6fe (origin/main, main, conflict-one) Merge pull request #2 from wongellakew-lab/conflict-one
| |
| | * 10db484 (origin/conflict-one) feat: update pass_threshold on conflict-one
| |
| | / 8fbd58c feat: update pass_threshold on conflict-two
| |
| | * 84c866c feat: finish PDF certificate logic
```

Exercise 5: The history cleanup (Safe Squash Workflow)

Simulate mess

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit --allow-empty -m "chore: started certificate logic"
[main a8b3cf1] chore: started certificate logic
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit --allow-empty -m "fix: forgot semicolon in PDF generator"
[main 389f90d] fix: forgot semicolon in PDF generator
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit --allow-empty -m "oops: typo in student name"
[main fcf7027] oops: typo in student name
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit --allow-empty -m "feat: finish PDF certificate logic"
[main 84c866c] feat: finish PDF certificate logic
```

Squash with reset --soft

Create a clean branch for the squash and reset the last 4 commits but keep all the changes staged and
Recommit as one clean commit

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git switch -c cleanup/certificate-generator
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git reset --soft HEAD~4
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git commit --allow-empty -m "feat(certificate): implement PDF certificate generation"
[cleanup/certificate-generator bf12e35] feat(certificate): implement PDF certificate generation
```

Verify

```
PS C:\Users\wonge\Desktop\FullStackDevelopment\Version Control> git log --oneline -5
bf12e35 (HEAD -> cleanup/certificate-generator) feat(certificate): implement PDF certificate generation
bc6235d Merge branch 'feature/strict-assessments' into main
74ced80 fix(assessment): lower threshold for inclusive grading
a9a4b8a (feature/strict-assessments) fix(assessment): increase threshold for honors certification
ff3b20d chore: setup baseline assessment rules
```

Reflections

What I learned from this module:

1. git reflog helps to see history of every HEAD move which will help to recover lost commits and to reset back to safety.
2. To use reset --soft instead of rebase. Editing with vim/nano during interactive rebase can be sometimes confusing so I learned: git reset --soft HEAD~3 keeps all changes staged, then I commit once cleanly.
3. Resolving conflicts in two places. I learned to fix conflicts locally with git merge main on my branch, stage, and commit. Also resolved directly on GitHub's web editor.
4. Learned new git commands like committing with no changes and displaying merges in a graph
5. Add and commit together. git commit -a -m "message" stages all tracked files and commits at once.